Relational Databases & SQL

Complete Reference Guide

Relational Databases · RDBMS · SQL vs NoSQL · SQL Keywords · Data Types · Operators

Table of Contents

1 Relational Databases

1.1 What is a Relational Database?

A **relational database** is a type of database that organizes data into one or more **tables** (also called *relations*), where data is structured in **rows** (records) and **columns** (attributes). Each table represents a specific entity (e.g., Customers, Orders, Products), and tables are linked together through **keys** to represent real-world relationships.

Introduced by **Edgar F. Codd** at IBM in 1970, the relational model is based on mathematical set theory and predicate logic. It remains the most widely used database model in the world due to its simplicity, flexibility, and strong consistency guarantees.

Example — Customers Table:

CustomerID	FirstName	LastName	Email	City
101	Alice	Sharma	alice@example.com	Mumbai
102	Bob	Patel	bob@example.com	Delhi
103	Carol	Rao	carol@example.com	Chennai

1.2 Core Concepts & Terminology

Table (Relation)	The fundamental storage unit. A two-dimensional structure with rows and columns.
Row (Tuple / Record)	A single entry in a table representing one instance of an entity. E.g., one customer.
Column (Attribute / Field)	A named property of an entity. Each column has a defined data type (INT, VARCHAR, DATE...).
Primary Key (PK)	A column (or set of columns) that uniquely identifies every row in a table. Must be unique and NOT NULL.
Foreign Key (FK)	A column in one table that references the Primary Key of another table, establishing a relationship.
Schema	The logical blueprint of a database — defines tables, columns, data types, constraints, and relationships.
Index	A data structure that speeds up data retrieval. Like a book index — avoids full table scans.
Constraint	Rules enforced on columns: NOT NULL, UNIQUE, CHECK, DEFAULT, PRIMARY KEY, FOREIGN KEY.

Normalization	The process of organizing a database to reduce redundancy and improve data integrity (1NF → 2NF → 3NF → BCNF).
ACID Properties	Atomicity, Consistency, Isolation, Durability — the guarantees that make database transactions reliable.

1.3 How RDBMS Works

A **Relational Database Management System (RDBMS)** is the software layer that manages relational databases. It handles storage, retrieval, security, and concurrency. Here is how queries flow through an RDBMS:

- **1. SQL Query Submitted:** The user or application sends an SQL statement to the RDBMS engine.
- **2. Parser & Tokenizer:** The engine parses the SQL text, checks syntax, and converts it into an internal parse tree.
- **3. Query Optimizer:** The optimizer evaluates multiple execution plans and picks the most efficient one based on indexes and statistics.
- **4. Execution Engine:** The chosen plan is executed. The engine reads/writes data pages from the buffer pool (memory cache).
- **5. Storage Engine:** If data is not in the buffer pool, it reads from disk files. On writes, changes are logged for crash recovery.
- **6. Result Returned:** The result set is assembled and returned to the client application.

2 RDBMS — Benefits & Limitations

2.1 Benefits of RDBMS

✓ Data Integrity

Constraints (PK, FK, UNIQUE, CHECK) and ACID transactions guarantee that data remains accurate and consistent even during failures or concurrent access.

✓ Structured Query Language

SQL is a declarative, standardized, and powerful language understood by virtually every RDBMS. It is easy to learn and highly expressive.

✓ Reduced Redundancy

Normalization eliminates duplicate data by splitting information into separate related tables, saving storage and preventing update anomalies.

✓ Flexible Querying

Complex queries with JOINs, aggregations, subqueries, and window functions can answer virtually any business question without changing the schema.

✓ Security

Fine-grained access control through GRANT/REVOKE statements. Row-level and column-level security in modern RDBMS.

✓ Concurrency Control

Multi-Version Concurrency Control (MVCC) and locking mechanisms allow thousands of users to access data simultaneously without conflicts.

✓ Data Independence

Physical changes (adding indexes, moving files) do not affect application code. Logical changes are also isolated through views.

✓ Mature Ecosystem

Decades of tooling, optimization techniques, backup solutions, monitoring systems, and skilled professionals are available.

✓ Referential Integrity

Foreign keys ensure that relationships between tables are always valid — no orphaned records.

✓ Backup & Recovery

RDBMS provides point-in-time recovery, transaction logs, and hot backup capabilities to ensure business continuity.

2.2 Limitations of RDBMS

✗ Scalability Challenges	RDBMS scales vertically (bigger hardware) better than horizontally (more servers). Sharding is complex and costly.
✗ Fixed Schema	Schema must be defined upfront. Altering tables in production (especially large ones) is slow and risky.
✗ Complex Hierarchical Data	Storing tree-structured or deeply nested data (like JSON documents or graphs) is cumbersome with relational tables.
✗ Performance at Scale	Joins across very large tables with billions of rows can be slow even with indexes, requiring expensive query tuning.
✗ High Cost (Commercial)	Enterprise RDBMS like Oracle and SQL Server carry significant licensing costs for large deployments.
✗ Impedance Mismatch	Object-oriented programming languages use objects, but RDBMS uses tables. ORMs add a mapping layer that introduces overhead.
✗ Limited Unstructured Data	RDBMS is not well-suited for storing images, videos, logs, and other unstructured or semi-structured data natively.

2.3 Popular RDBMS Products

RDBMS	Vendor	Type	Best For
MySQL	Oracle	Open Source	Web apps, CMS, e-commerce
PostgreSQL	Community	Open Source	Complex queries, GIS, analytics
Oracle DB	Oracle	Commercial	Enterprise, banking, ERP
SQL Server	Microsoft	Commercial	Windows enterprise, .NET apps
SQLite	Community	Embedded/Free	Mobile apps, local storage, testing
MariaDB	Community	Open Source	MySQL drop-in replacement
IBM Db2	IBM	Commercial	Mainframe, enterprise analytics

3 SQL vs NoSQL Databases

3.1 What is SQL (Relational)?

SQL databases (also called **relational databases**) store data in structured tables with a predefined schema. They use **Structured Query Language (SQL)** for all data operations and enforce **ACID** compliance to guarantee reliability.

3.2 What is NoSQL (Non-Relational)?

NoSQL databases (*Not Only SQL*) are non-relational databases designed for flexibility, horizontal scalability, and high availability. They sacrifice strict consistency for performance and can handle **unstructured or semi-structured data** natively. NoSQL follows the **BASE** model: *Basically Available, Soft state, Eventually consistent*.

NoSQL Database Types:

Document Store	Stores JSON/BSON documents. Each document can have different fields. Example: **MongoDB**, CouchDB.
Key-Value Store	Simplest model — data stored as key:value pairs. Extremely fast for lookups. Example: **Redis**, DynamoDB.
Column-Family	Data stored in column families rather than rows. Great for analytics. Example: **Apache Cassandra**, HBase.
Graph Database	Stores nodes and edges for relationship-heavy data. Example: **Neo4j**, Amazon Neptune.
Time-Series DB	Optimized for time-stamped data (metrics, logs). Example: **InfluxDB**, TimescaleDB.
Search Engine	Full-text search optimized. Example: **Elasticsearch**, Apache Solr.

3.3 Detailed Comparison

Feature	SQL (Relational)	NoSQL (Non-Relational)
Data Model	Tables with rows and columns	Documents, key-value, columns, or graphs
Schema	Fixed, predefined schema	Dynamic / schema-less or flexible schema
Query Language	SQL — standardized	Database-specific APIs or query language
Scalability	Vertical (scale-up)	Horizontal (scale-out across many nodes)
Consistency	ACID — strong consistency	BASE — eventual consistency (usually)

Joins	Full JOIN support	Joins are rare; data is often denormalized
Transactions	Multi-table ACID transactions	Limited; most support single-doc atomicity
Best Use Cases	Finance, ERP, CRM, e-commerce	Big data, real-time apps, IoT, content
Examples	MySQL, PostgreSQL, Oracle, SQL Server	MongoDB, Redis, Cassandra, Neo4j, DynamoDB
Maturity	50+ years; highly mature	Emerged ~2009; rapidly evolving
Cost	Free (open source) to expensive	Often free/open source; managed = paid

3.4 When to Use Which

Use SQL when:

- You need strong data consistency and integrity
- Your data is structured and relationships matter
- You need complex queries, reporting, and analytics
- Regulatory compliance requires ACID transactions (banking, healthcare)

Use NoSQL when:

- You have rapidly changing or unpredictable data structures
- You need to scale horizontally across many servers
- You are handling big data, IoT sensor streams, or real-time feeds
- Your data is hierarchical, graph-based, or needs full-text search

4 SQL Keywords

SQL keywords are reserved words that perform specific database operations. They are grouped into five categories based on their purpose.

4.1 DDL — Data Definition Language

DDL commands define and modify the **structure** of database objects (tables, indexes, views). DDL statements are auto-committed — they cannot be rolled back.

CREATE — Creates a new database object (table, view, index, schema, database).

```
CREATE TABLE Students (
StudentID INT PRIMARY KEY,
Name VARCHAR(100) NOT NULL,
Age INT CHECK(Age >= 0),
Email VARCHAR(150) UNIQUE
);
```

ALTER — Modifies an existing table — add/drop/modify columns, rename, add constraints.

```
ALTER TABLE Students ADD COLUMN GPA DECIMAL(3,2);
ALTER TABLE Students MODIFY COLUMN Name VARCHAR(200);
ALTER TABLE Students DROP COLUMN Age;
```

DROP — Permanently deletes a database object and all its data.

```
DROP TABLE Students;
DROP DATABASE mydb;
DROP INDEX idx_email;
```

TRUNCATE — Removes all rows from a table instantly. Faster than DELETE; cannot be rolled back.

```
TRUNCATE TABLE Students;
```

RENAME — Renames a database object.

```
RENAME TABLE Students TO Learners;
```

COMMENT — Adds a comment to a table or column for documentation.

```
COMMENT ON TABLE Students IS 'Stores student enrollment data';
```

4.2 DML — Data Manipulation Language

DML commands manipulate the **data** stored inside tables. DML statements can be rolled back within a transaction.

INSERT — Adds new rows into a table.

```
INSERT INTO Students (StudentID, Name, Email)
VALUES (1, 'Alice Sharma', 'alice@uni.edu');

-- Insert multiple rows
INSERT INTO Students VALUES
(2, 'Bob Patel', 22, 'bob@uni.edu'),
(3, 'Carol Rao', 20, 'carol@uni.edu');
```

UPDATE — Modifies existing rows. Always use WHERE to avoid updating all rows.

```
UPDATE Students
SET GPA = 3.8, Email = 'alice_new@uni.edu'
WHERE StudentID = 1;
```

DELETE — Removes specific rows. Without WHERE it deletes all rows (but is logged and rollback-able).

```
DELETE FROM Students WHERE StudentID = 3;

-- Delete all rows (rollback-able, unlike TRUNCATE)
DELETE FROM Students;
```

MERGE — Upsert operation — INSERT if not exists, UPDATE if exists. (UPSERT in some databases).

```
MERGE INTO Students AS target
USING NewStudents AS source
ON target.StudentID = source.StudentID
WHEN MATCHED THEN
UPDATE SET target.GPA = source.GPA
WHEN NOT MATCHED THEN
INSERT VALUES (source.StudentID, source.Name, source.Email);
```

4.3 DQL — Data Query Language

DQL consists primarily of the **SELECT** statement, used to retrieve data. It is the most frequently used SQL category and supports powerful clauses.

```
SELECT s.Name, s.GPA, d.DeptName
FROM Students s
JOIN Departments d ON s.DeptID = d.DeptID
WHERE s.GPA >= 3.5
AND d.DeptName IN ('Science', 'Engineering')
GROUP BY d.DeptName
HAVING COUNT(*) > 5
ORDER BY s.GPA DESC
LIMIT 10 OFFSET 20;
```

SELECT	Specifies which columns to retrieve. Use * for all columns or list specific ones.
FROM	Specifies the table(s) to query from.
JOIN	Combines rows from two or more tables. Types: INNER, LEFT, RIGHT, FULL OUTER, CROSS, SELF.
WHERE	Filters rows before any grouping. Applies to individual row conditions.
GROUP BY	Groups rows that have the same values in specified columns (for aggregation).
HAVING	Filters groups after GROUP BY. Works like WHERE but on aggregated results.
ORDER BY	Sorts the result set. ASC (default) or DESC. Can sort by multiple columns.
LIMIT/TOP	Restricts the number of rows returned. LIMIT in MySQL/PostgreSQL; TOP in SQL Server.
DISTINCT	Removes duplicate rows from the result set.
ALIAS (AS)	Renames a column or table in the output for readability.

4.4 DCL — Data Control Language

DCL manages **access permissions** and security within the database.

GRANT — Gives specific privileges to a user or role.

```
GRANT SELECT, INSERT ON Students TO 'teacher_user'@'localhost';
GRANT ALL PRIVILEGES ON mydb.* TO 'admin'@'%';
```

REVOKE — Removes previously granted privileges.

```
REVOKE INSERT ON Students FROM 'teacher_user'@'localhost';
```

4.5 TCL — Transaction Control Language

TCL commands manage **transactions** — groups of SQL statements that must all succeed or all fail together, ensuring ACID compliance.

BEGIN / START TRANSACTION — Marks the start of a transaction block.

```
START TRANSACTION;
UPDATE Accounts SET Balance = Balance - 500 WHERE AccID = 1;
UPDATE Accounts SET Balance = Balance + 500 WHERE AccID = 2;
```

COMMIT — Saves all changes made during the current transaction permanently.

```
COMMIT; -- Both UPDATE statements above are now permanent
```

ROLLBACK — Undoes all changes made since the last COMMIT or SAVEPOINT.

```
ROLLBACK; -- Reverts both UPDATE statements if something failed
```

SAVEPOINT — Creates a named checkpoint within a transaction for partial rollback.

```
SAVEPOINT sp1;
-- do some work
ROLLBACK TO SAVEPOINT sp1; -- undo only since sp1
```

5 SQL Data Types

Data types define the kind of value a column can store. Choosing the right data type is critical for performance, storage efficiency, and data integrity.

5.1 Numeric Data Types

Type	Storage	Range / Precision	Use Case
TINYINT	1 byte	-128 to 127 (or 0–255 UNSIGNED)	Flags, small counters, age
SMALLINT	2 bytes	-32,768 to 32,767	Small IDs, quantities
INT / INTEGER	4 bytes	-2,147,483,648 to 2,147,483,647	Primary keys, general numbers
BIGINT	8 bytes	-9.2 x 10^18 to 9.2 x 10^18	Large IDs, timestamps, big counts
DECIMAL(p,s)	Variable	Exact precision. p=total digits, s=decimals	Money, financial calculations
NUMERIC(p,s)	Variable	Same as DECIMAL	Same as DECIMAL
FLOAT	4 bytes	~7 decimal digits precision	Scientific data, approximate values
DOUBLE	8 bytes	~15 decimal digits precision	High-precision scientific data
BIT(n)	n bits	0 or 1 per bit	Boolean flags, bit masks

***Note:** Use **DECIMAL** for money — never FLOAT or DOUBLE, which can introduce rounding errors. For example, 0.1 + 0.2 in FLOAT may give 0.30000000000000004.*

5.2 String / Character Data Types

Type	Max Size	Padding	Use Case
CHAR(n)	255 chars	Fixed-length, right-padded with spaces	Fixed codes: country code, gender flag
VARCHAR(n)	65,535 chars	Variable-length, no padding	Names, emails, addresses
TEXT	65,535 chars	Variable, stored off-page if large	Descriptions, comments, articles
MEDIUMTEXT	16 MB	Variable	Blog posts, HTML content
LONGTEXT	4 GB	Variable	Full book text, large logs

ENUM(...)	1-2 bytes	Stores as integer internally	Status fields: 'active','inactive'
SET(...)	1-8 bytes	Bitmask of allowed values	Multiple-choice fields
NVARCHAR(n)	Platform dep.	Unicode variable-length	Multilingual content (UTF-16)

5.3 Date & Time Data Types

Type	Format	Range	Use Case
DATE	YYYY-MM-DD	1000-01-01 to 9999-12-31	Birth dates, event dates
TIME	HH:MM:SS	-838:59:59 to 838:59:59	Opening hours, durations
DATETIME	YYYY-MM-DD HH:MM:SS	1000-01-01 to 9999-12-31	Order timestamps, logging
TIMESTAMP	YYYY-MM-DD HH:MM:SS	1970-01-01 to 2038-01-19	Auto-updated row timestamps
YEAR	YYYY	1901 to 2155	Graduation year, model year
INTERVAL	Platform specific	Period of time	Differences, scheduling

> ***Tip: TIMESTAMP*** *stores in UTC and auto-converts to the session timezone.* ***DATETIME*** *stores exactly as entered with no timezone conversion. Use TIMESTAMP for audit columns (created_at, updated_at) and DATETIME for user-visible event times.*

5.4 Boolean & Binary Data Types

Type	Size	Description	Use Case
BOOLEAN / BOOL	1 byte	TRUE or FALSE (stored as 1 or 0 in MySQL)	Flags, active status
BINARY(n)	n bytes	Fixed-length binary string	Hash values, UUIDs
VARBINARY(n)	0-n bytes	Variable-length binary string	Encrypted data, raw bytes
BLOB	64 KB	Binary Large Object — stores binary data	Small images, files
MEDIUMBLOB	16 MB	Medium binary large object	Documents, audio clips
LONGBLOB	4 GB	Large binary large object	Videos, large files
JSON	Variable	Native JSON document storage (MySQL 5.7+, PG 9.2+)	Config, flexible attributes

UUID	16 bytes	Universally unique identifier	Distributed primary keys

6 SQL Operators

SQL operators are symbols or keywords used in expressions to perform operations on data — arithmetic calculations, comparisons, logical tests, and more.

6.1 Arithmetic Operators

Perform mathematical calculations on numeric values. Used in SELECT expressions, WHERE clauses, and UPDATE SET clauses.

Operator	Name	Example	Result
+	Addition	SELECT 10 + 3;	13
-	Subtraction	SELECT 10 - 3;	7
*	Multiplication	SELECT 10 * 3;	30
/	Division	SELECT 10 / 3;	3.3333...
%	Modulo	SELECT 10 % 3;	1
DIV	Integer Division	SELECT 10 DIV 3;	3

```
SELECT ProductName, Price, Quantity,
Price * Quantity AS TotalValue,
Price * 0.9 AS DiscountedPrice
FROM Products WHERE Price * Quantity > 1000;
```

6.2 Comparison Operators

Compare two values and return TRUE, FALSE, or NULL. Used primarily in WHERE, HAVING, and JOIN conditions.

Operator	Meaning	Example	Returns TRUE when...
=	Equal to	WHERE Age = 25	Age is exactly 25
<> / !=	Not equal to	WHERE Status != 'inactive'	Status is not 'inactive'
>	Greater than	WHERE Salary > 50000	Salary exceeds 50000
<	Less than	WHERE GPA < 2.0	GPA is below 2.0
>=	Greater than or equal	WHERE Age >= 18	Age is 18 or older
<=	Less than or equal	WHERE Stock <= 10	Stock is 10 or fewer

<=>	NULL-safe equal (MySQL)	WHERE x <=> NULL	Both sides are NULL or equal

6.3 Logical Operators

Combine multiple conditions in WHERE and HAVING clauses.

AND — Both conditions must be TRUE.

```
SELECT * FROM Employees
WHERE Department = 'IT' AND Salary >= 60000;
```

OR — At least one condition must be TRUE.

```
SELECT * FROM Students
WHERE Grade = 'A' OR GPA >= 3.7;
```

NOT — Negates a condition (TRUE becomes FALSE).

```
SELECT * FROM Orders
WHERE NOT Status = 'Cancelled';
```

ALL — TRUE if all values in a subquery satisfy the condition.

```
SELECT Name FROM Products
WHERE Price > ALL (SELECT Price FROM Products WHERE Category='Budget');
```

ANY/SOME — TRUE if any value in a subquery satisfies the condition.

```
SELECT Name FROM Products
WHERE Price > ANY (SELECT Price FROM Products WHERE Category='Budget');
```

EXISTS — TRUE if the subquery returns at least one row.

```
SELECT CustomerName FROM Customers c
WHERE EXISTS (
SELECT 1 FROM Orders o WHERE o.CustomerID = c.CustomerID
);
```

6.4 Special Operators

BETWEEN ... AND ... — Checks if a value falls within an inclusive range (works on numbers, dates, and strings).

```
SELECT * FROM Orders
WHERE OrderDate BETWEEN '2024-01-01' AND '2024-12-31';

SELECT * FROM Products WHERE Price BETWEEN 100 AND 500;
```

IN (...) — Checks if a value matches any value in a provided list or subquery. Equivalent to multiple OR conditions.

```
SELECT * FROM Employees
WHERE Department IN ('HR', 'Finance', 'IT');

SELECT * FROM Products
WHERE CategoryID IN (SELECT ID FROM Categories WHERE Active = 1);
```

LIKE — Pattern matching for strings. % matches zero or more characters; _ matches exactly one character.

```
SELECT * FROM Customers WHERE Name LIKE 'A%'; -- starts with A
SELECT * FROM Products WHERE SKU LIKE '___-2024'; -- 3 chars, dash, 2024
SELECT * FROM Emails WHERE Address LIKE '%@gmail.com';
```

IS NULL / IS NOT NULL — Tests for NULL values. Note: = NULL does NOT work — always use IS NULL.

```
SELECT * FROM Employees WHERE ManagerID IS NULL; -- top-level managers
SELECT * FROM Students WHERE GPA IS NOT NULL;
```

CASE WHEN ... THEN ... END — Conditional expression — SQL's equivalent of if-else. Can be used in SELECT, WHERE, ORDER BY.

```
SELECT Name, GPA,
CASE
WHEN GPA >= 3.7 THEN 'Distinction'
WHEN GPA >= 3.0 THEN 'Merit'
WHEN GPA >= 2.0 THEN 'Pass'
ELSE 'Fail'
END AS Grade
FROM Students;
```

COALESCE(a, b, ...) — Returns the first non-NULL value in the list. Extremely useful for handling NULLs.

```
SELECT COALESCE(MiddleName, '') AS MiddleName FROM Persons;
SELECT COALESCE(Phone, Mobile, Email, 'No contact') AS Contact FROM Users;
```

NULLIF(a, b) — Returns NULL if a equals b; otherwise returns a. Prevents division-by-zero errors.

```
SELECT TotalSales / NULLIF(NumTransactions, 0) AS AvgSale FROM Summary;
```

6.5 Set Operators

Set operators combine the results of two or more SELECT statements. Both queries must return the same number of columns with compatible data types.

UNION — Combines results from two queries and removes duplicates.

```
SELECT Name FROM Employees
UNION
SELECT Name FROM Contractors; -- no duplicates
```

UNION ALL — Combines results from two queries keeping ALL rows including duplicates. Faster than UNION.

```
SELECT ProductID FROM Sales_2023
UNION ALL
SELECT ProductID FROM Sales_2024; -- all rows, including duplicates
```

INTERSECT — Returns only rows that appear in BOTH result sets.

```
SELECT CustomerID FROM OnlineOrders
INTERSECT
SELECT CustomerID FROM StoreOrders; -- customers who bought from both
```

EXCEPT / MINUS — Returns rows from the first query that do NOT appear in the second. (MINUS in Oracle)

```
SELECT StudentID FROM AllStudents
EXCEPT
SELECT StudentID FROM GraduatedStudents; -- students not yet graduated
```

Quick Reference Summary

Category	Key Points
Relational DB	Tables, rows, columns; linked via keys; ACID properties; managed by RDBMS
DDL Commands	CREATE, ALTER, DROP, TRUNCATE, RENAME — define structure
DML Commands	INSERT, UPDATE, DELETE, MERGE — manipulate data
DQL Command	SELECT with FROM, JOIN, WHERE, GROUP BY, HAVING, ORDER BY, LIMIT
DCL Commands	GRANT, REVOKE — manage user access
TCL Commands	COMMIT, ROLLBACK, SAVEPOINT — manage transactions
Numeric Types	TINYINT, INT, BIGINT, DECIMAL (use for money), FLOAT, DOUBLE
String Types	CHAR (fixed), VARCHAR (variable), TEXT, ENUM, JSON
Date/Time Types	DATE, TIME, DATETIME, TIMESTAMP, YEAR
Operators	Arithmetic (+,-,*,/,%), Comparison (=,<>,>,<,>=,<=), Logical (AND,OR,NOT,EXISTS)
Special Ops	BETWEEN, IN, LIKE, IS NULL, CASE WHEN, COALESCE, NULLIF
Set Operators	UNION, UNION ALL, INTERSECT, EXCEPT
SQL vs NoSQL	SQL: structured, ACID, fixed schema. NoSQL: flexible, horizontal scale, BASE